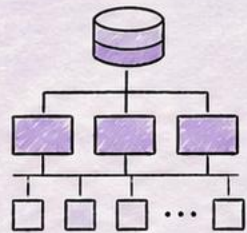
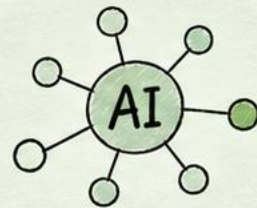


项目背景

建设思路



当前架构

SmartPerfetto 技术交流

AI-assisted Android Perfetto 分析平台

问题场景



未来计划

双方讨论



1

项目介绍



2

对方分享

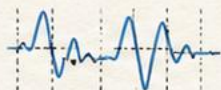


3

开放讨论



2026-06-16



把 trace 里的事实，变成能被复查的结论。





高建武 / Gracker

Android Performance • SmartPerfetto

📍 目前：成都 MTK 性能优化团队

🎯 方向：Android Framework /
APM / Perfetto / Systrace



经历

乐蛙 • 魅族 • OPPO • OnePlus



作品

SmartPerfetto • TraceFix • 100Years
Android App Memory Analysis • Perfetto Auto-Pin



核心 AI 工具

Codex • Claude Code • Openclaw



关注

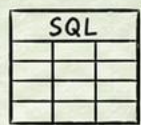
把 Trace 分析经验做成可复查的工具和 AI 分析流程

为什么要做 SmartPerfetto

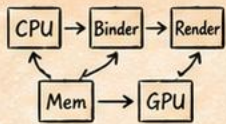
现实困境



数据太大
50-500MB trace
百万级事件



计算要准
P50 / P90 / P99
帧耗时、Binder 等待
CPU 频率、线程状态



机制交叉
启动、渲染、调度、内存、
SurfaceFlinger、HWC/GPU
互相影响



人工成本高
依赖经验看 timeline 和 SQL
结论难复盘、难统一



LLM 边界
直接喂数据上下文不够
容易编数字、遗漏关键轨道

SmartPerfetto

AI-assisted Android
Perfetto 分析平台

我们要达到



工具取数
通过 SQL / Skill
拿到小而准的数据



AI 归因
基于证据做因果解释
提供可行动建议



证据可复查
来源清晰、逻辑可追踪
结论可被复查验证



工具取数
SQL / Skill
精确统计



AI 归因
理解问题
分析计划
因果解释



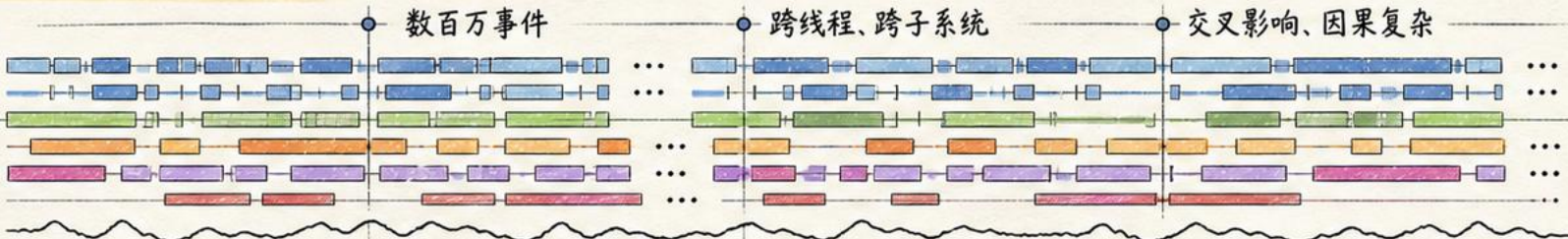
报告输出
多维证据
标准结构
可复盘



证据可复查
来源可追溯
结果可验证
质量可控

Perfetto trace

CPU 0
CPU 1
RenderThread
Binder Thread
SurfaceFlinger
GPU
Memory

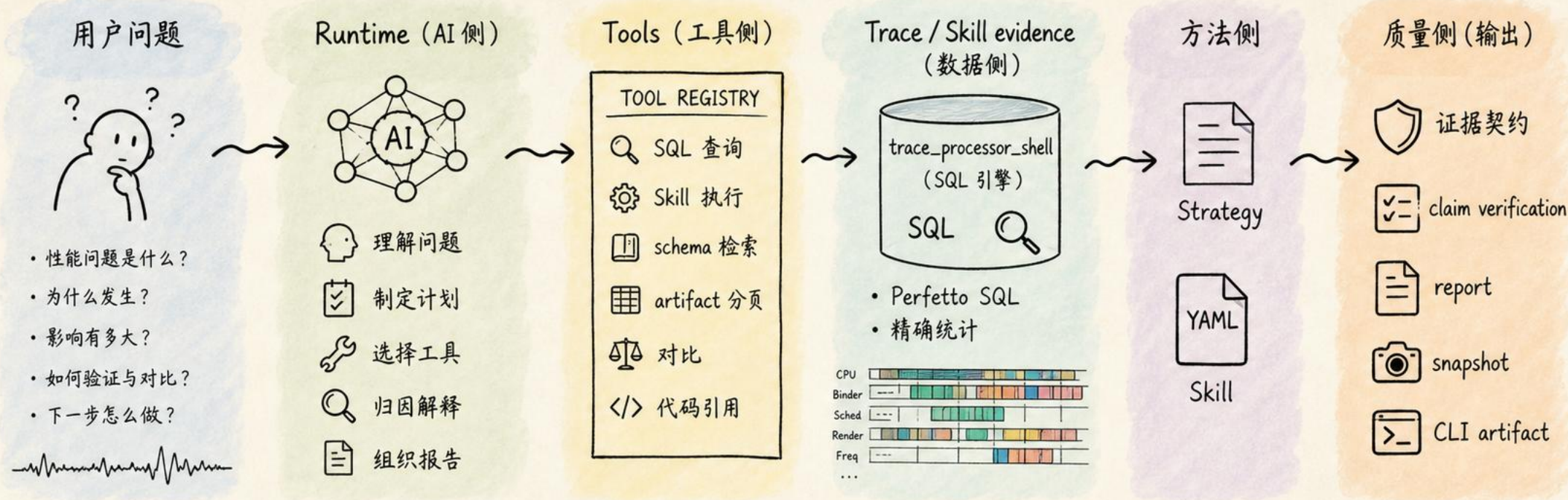


Perfetto 给事实, SmartPerfetto 负责把事实组织成可复查判断。

需要解决的核心

- ✓ 规模可承载
- ✓ 统计要精确
- ✓ 归因要合理
- ✓ 证据可追溯
- ✓ 结论可复查

建设思路：让模型只做擅长的部分



raw trace (原始数据)

1010101010
1010010101
0101010101

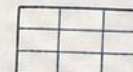
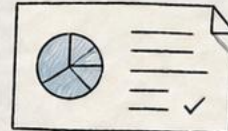


evidence (证据与统计)



...

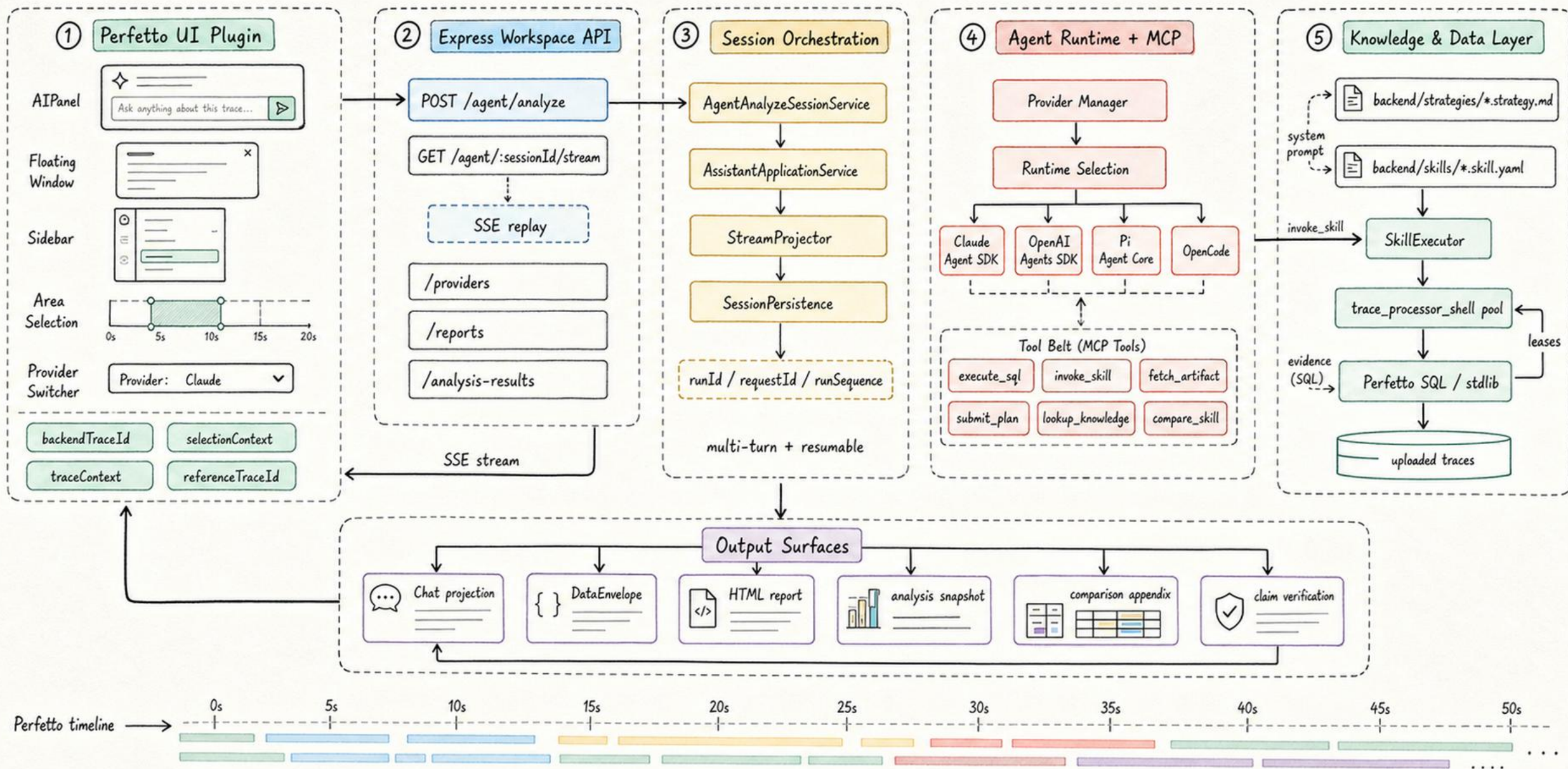
report (报告与结论)

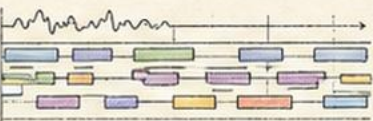
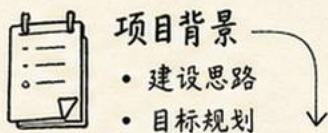


* 模型不猜数字，数字来自工具。

SmartPerfetto 前后端架构

前端收集上下文，后端完成证据闭环



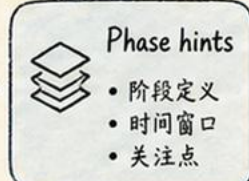
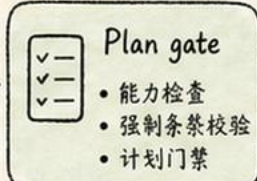
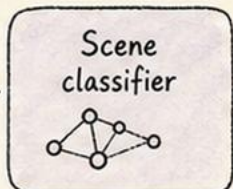
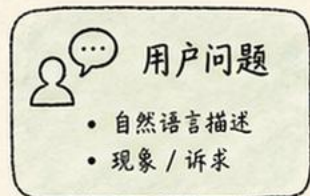
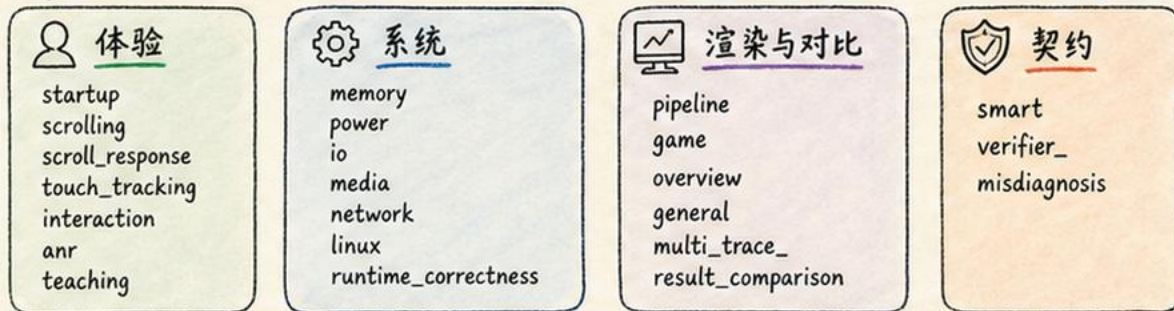


Strategy: 场景级分析契约

21 个 Strategy 文件
19 normal · 2 contract-only

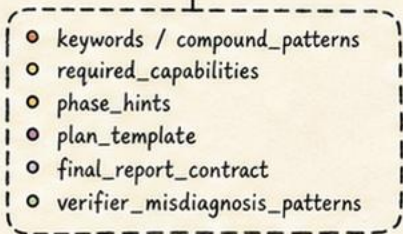
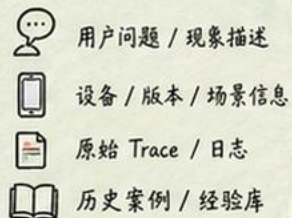
backend/strategies/*.strategy.md

当前 Strategy 清单 (按场景分组)



分析完成

输入与信号

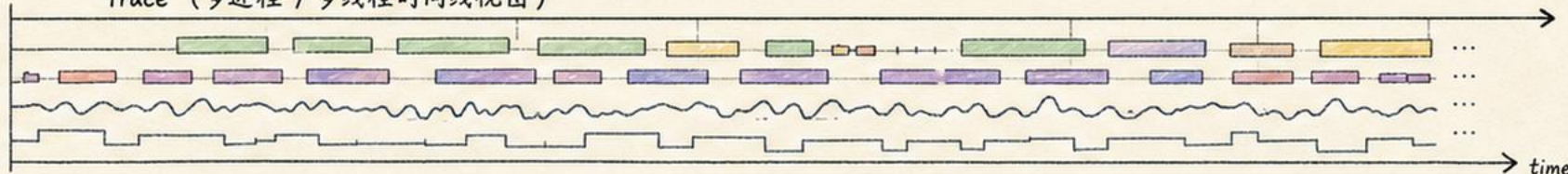


产出与沉淀



Trace (多进程 / 多线程时间线视图)

U Thread
RenderThread
Binder Thread
CPU Freq
Power Rails
...



相关产物



Strategy 管分析是否充分; Skill 管证据如何取得

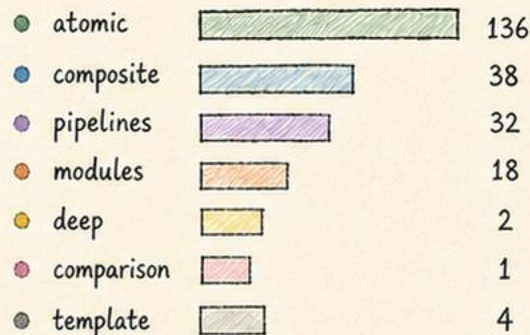
231 个 .skill.yaml

Skill: 可执行取证能力库

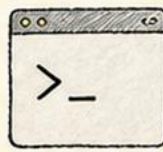


来源: backend/skills/**/*/.skill.yaml

当前 Skill 存量 (按层级)



invoke_skill



CLI / API / UI

SkillExecutor



- typed inputs
- prerequisites / Perfetto modules
- SQL fragments
- condition / iterator / parallel
- display columns
- synthesize / artifact

DataEnvelope



- inputs
- params
- trace ctx
- results
- metadata

Artifact / Report / Snapshot



为什么需要 Skill

- 原子能力可复用
- 组合能力覆盖场景
- 深度能力支撑定位
- 保证证据可复查

Skill 放什么

- typed inputs
- SQL / 子 Skill / 条件分支
- display / artifact / DataEnvelope

Composite (场景组合能力)

- startup_analysis
- scrolling_analysis
- jank_frame_detail
- anr_analysis
- binder_analysis
- memory_analysis
- cpu_analysis
- gpu_analysis
- ...

Atomic (原子取证探针)

- consumer_jank_detection
- binder_root_cause
- frame_outrun_summary
- cpu_time_per_frame
- input_to_frame_latency
- vrr_detection
- ...

Pipelines (渲染 / 管线能力)

- android_view_standard_blast
- compose_standard
- flutter_textureview
- webview_gl_funcutor
- rn_new_arch
- vulkan_native
- ...

Modules / Deep (模块 / 深度能力)

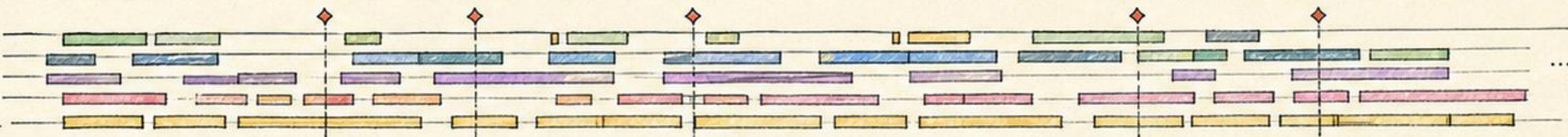
- scheduler_module
- binder_module
- surfaceflinger_module
- input_module
- cpu_module
- gpu_module
- ...
- callstack_analysis
- cpu_profiling
- ...



Perfetto Trace timeline

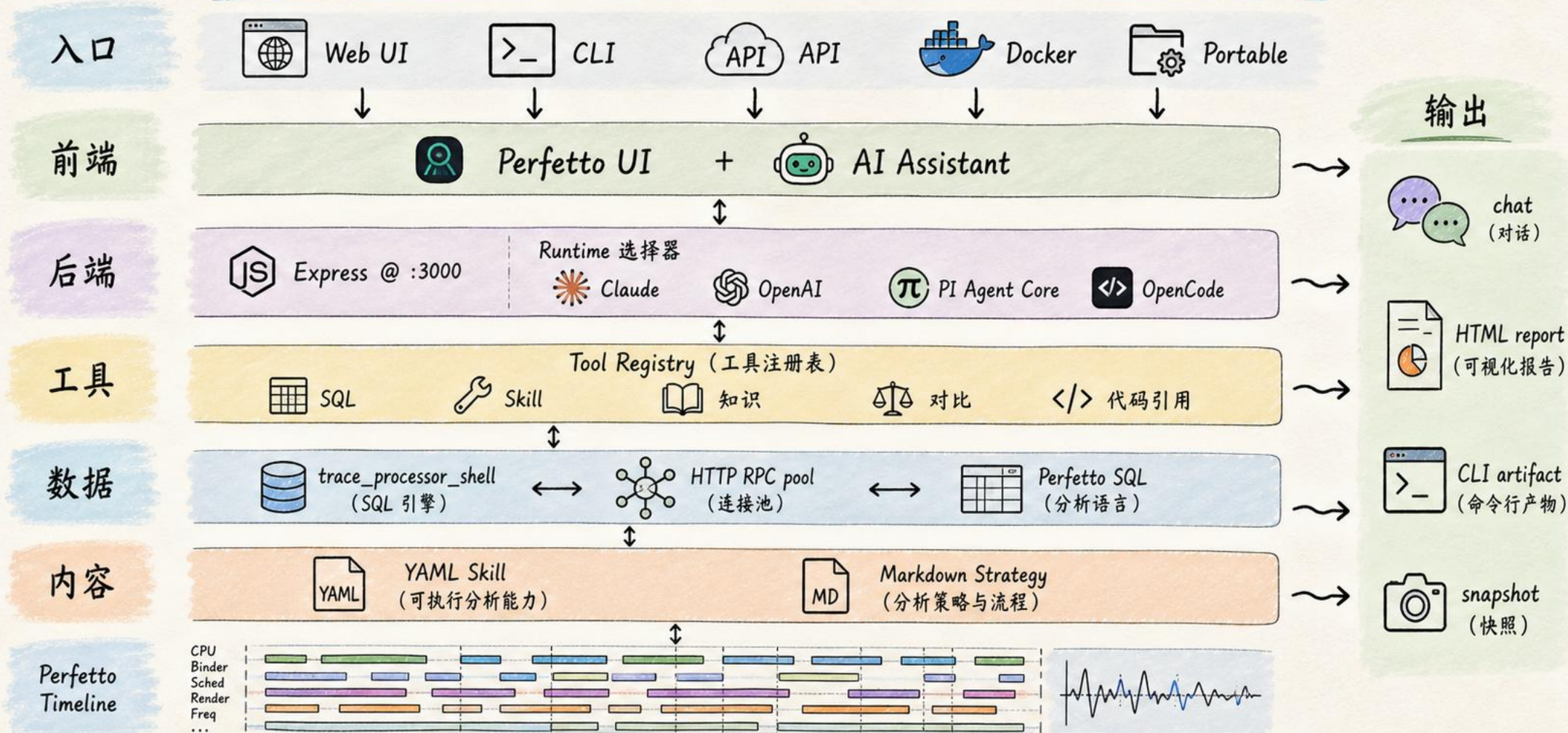
track

CPU 0
CPU 1
GPU
Binder
SurfaceFlinger



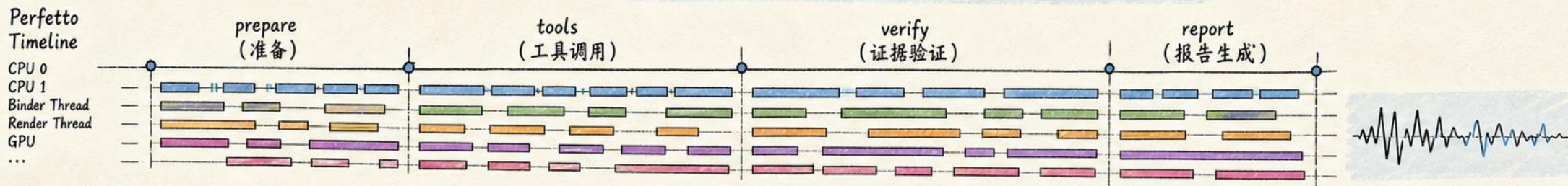
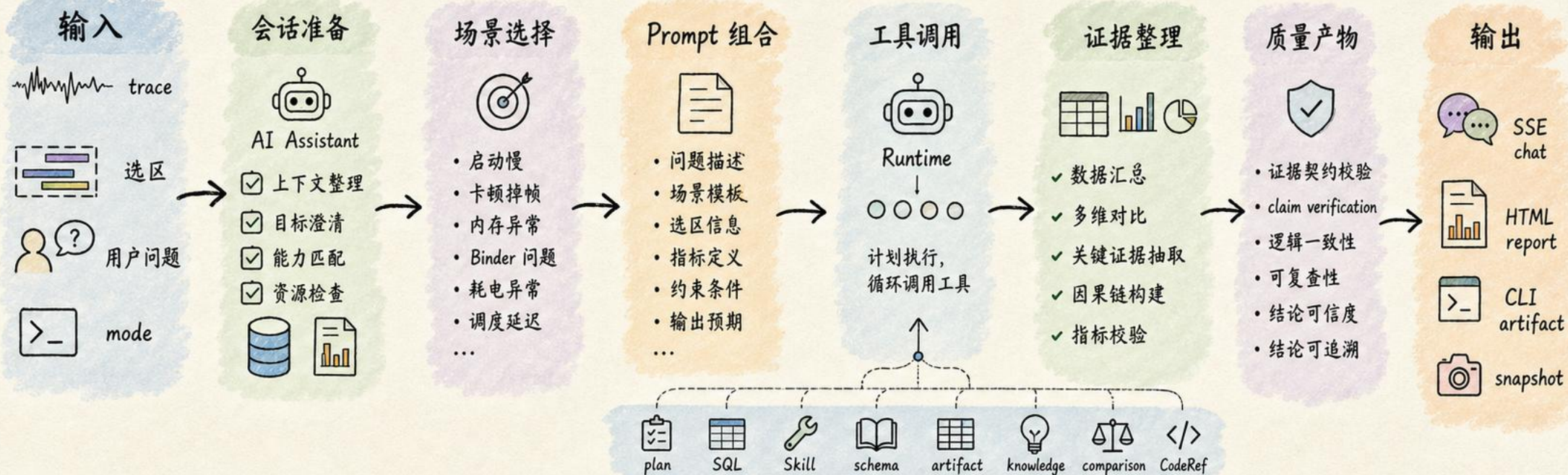
一个 invoke_skill 背后, 是可复查的数据管线

当前架构：Perfetto UI 之上的 AI 分析层



★ 同一套分析能力，服务 UI、CLI、API 和报告。

一次分析如何从问题走到报告



分析不是一段回答，而是一组可追溯产物。



证据



关联



结论



可复查

SmartPerfetto 现在解决哪些场景



① 启动性能

- 首帧时间 (TTFP)
- 首屏渲染完成
- 冷启动 / 热启动

② 滑动/Jank



- 慢帧定位与归因
- 掉帧区间 & 卡顿来源

③ ANR/交互



- 主线程阻塞分析
- 输入事件链路追踪
- Binder 等待链

④ CPU/内存/功耗



- CPU 热点 & 调度
- 内存分配 & LMK
- 功耗区间归因

⑤ 渲染管线



- SurfaceFlinger 分析
- 合成延迟 & 同步栅栏
- 丢帧 & 提交异常

⑥ 对比



Metric	A	B	Δ
			↑
			↓

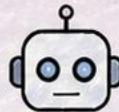
- 多版本 / 多设备对比
- 关键指标 delta table
- 回归定位

⑦ Code-aware

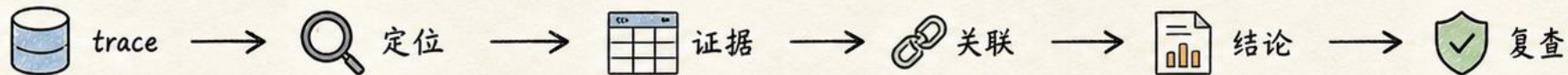


- 函数栈还原 & 聚类
- 关键路径 CodeRef
- 源码行级定位

⑧ 自动化



- 脚本化分析流程
- CI 回归门禁
- CLI/API 输出

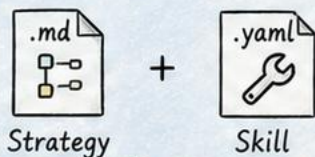


★ 场景越复杂，越需要把证据路径固定下来。

几个影响架构的取舍

① 内容驱动

Strategy + Skill
把分析能力外置为文件，
让演进可控、可版本化



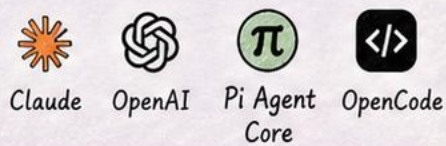
② 工具注册

统一的工具注册表，
降低耦合，便于扩展



③ Runtime 可替换

模型与执行内核可替换，
避免平台绑定



④ 输出分面

同一次分析，多种输出分面，
满足不同使用场景



内容
Strategy
+ Skill



工具
注册表
(Tool Registry)



Runtime
执行与生成



输出
多分面产物



artifact store
统一产物存储
便于追溯与复查

⑤ 上下文节省

严格的 request scope，
只给模型需要的上下文



- ✓ 目标明确
- ✓ 选择性加载
- ✓ 减少噪声

⑥ 证据约束

以证据为中心，契约化约束，
让结论可验证、可复查



证据提取



claim verification
(契约校验)



结论产出

- ✓ 证据契约
- ✓ 一致性校验
- ✓ 可复查追溯

⑦ 多入口一致

Web UI / CLI / API 一致访问，
共享同一套能力与产物



Web UI



CLI



API

- ✓ 同一能力
- ✓ 同一契约
- ✓ 同一产物



把变化放到文件，把契约留给系统。



做起来难在哪里

① 数据规模



- 50~500MB trace
- 百万级事件
- 多设备 / 长时采集
- 查询开销与内存峰值



挑战: 高效查询、
分片与缓存策略、
稳定性保障

② 身份解析

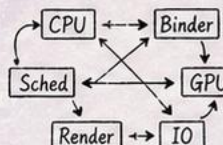


- PID/TID 重用
- 线程名不稳定
- UID/包名漂移
- Binder 集成解析



挑战: 统一身份模型,
稳定、可追溯的映射

③ 跨子系统归因



- 因果链长、跨域
- 时钟源与对齐
- 事件含义差异



挑战: 构建可靠因果链,
降低误判与遗漏

④ Runtime 差异



Claude



OpenAI



Pi Agent Core



OpenCode

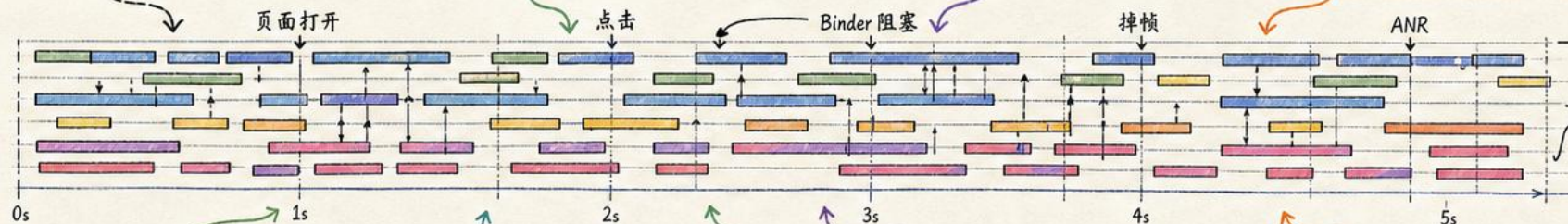
- 能力边界不同
- 推理风格不同
- 函数调用一致性
- 成本 & 速率限制



挑战: 抽象契约统一,
稳定、可替换、可观测

Perfetto trace

CPU 0
CPU 1
Binder Thread
Render Thread
GPU
SurfaceFlinger
Freq
...



多线程、多进程、
多子系统交织
证据链长且脆弱,
任一环节缺失会
导致误判

⑤ 输出面多



chat



HTML report



CLI artifact



snapshot

- 同源内容多种形态
- 实时 vs 批处理
- 一致性与可追溯性
- 大报告与分页策略



挑战: 内容模型统一,
多面输出一致可复现

⑥ Code-aware 隐私



CodeRef

- 源码路径解析
- 代码片段引用
- 仓库权限与隔离
- 隐私与合规风险



挑战: 最小化暴露,
可控引用与脱敏策略

⑦ 分发边界



Docker



CLI



portable

- 环境依赖复杂
- 数据与插件分发
- 版本兼容性
- provider pinning



挑战: 可移植、可离线、
依赖锁定与升级策略

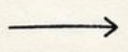
⑧ 验证成本



- 结果可信度
- 场景覆盖难
- 数据集维护成本
- scene regression
- 复查与根因闭环



挑战: 自动化验证,
可复现、可复查闭环



分析 → 证据 (artifact) → 结论 → 复查 → 归档/回归 (scene regression)

☆ AI 只是入口, 工程难度在证据、边界和复查。

后续计划与讨论问题

路线图



讨论



采集怎么做？



指标怎么比？



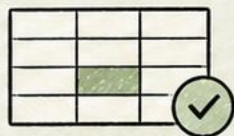
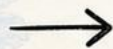
隐私怎么控？



哪些场景可共建？



trace



evidence cell



claim verification



结论



下一步不是多说结论，而是让结论更容易被验证。